UNIVERSIDAD DE PALERMO

Microservicios y APIs Escalables · 2026

SPRINT 9

Arquitectura Hexagonal y Descomposición en Microservicios

Fase 2 — Microservicios (Sprints 9-14)
Clases 9-10 | Duración estimada: 2 sprints
Entregable: repositorio multi-módulo con ms-dueno en arquitectura hexagonal funcionando
Stack: Spring Boot 4.1.0 · Spring Data JPA · MySQL · Maven multi-módulo
Patrón central: Hexagonal Architecture (Ports & Adapters)

1. Objetivo del Sprint

Hasta ahora trabajaron con un monolito: una sola aplicación Spring Boot con todas las entidades, repositorios, servicios y controladores juntos. En este sprint hacemos la transición conceptual y estructural hacia microservicios.

Al finalizar este sprint van a poder:

- Explicar cuándo y por qué conviene pasar de monolito a microservicios.
- Identificar los Bounded Contexts del dominio veterinario.
- Estructurar un proyecto Maven multi-módulo con arquitectura hexagonal.
- Implementar ms-dueno completo con las tres capas hexagonales.

2. Monolito vs. Microservicios — ¿Cuándo y por qué?

El monolito que construyeron es perfectamente válido para un proyecto inicial. A medida que el sistema crece, aparecen limitaciones reales:

- Un cambio en la entidad Mascota obliga a re-deployar toda la aplicación.
- Si el módulo de Turnos tiene picos de carga, no podemos escalar solo eso.
- Un bug en el contexto de Veterinarios puede tirar el contexto de Dueños.

Aspecto	Monolito	Microservicios
Deploy	Todo junto — un .jar	Servicios independientes
Escalado	Toda la app o nada	Por servicio según demanda
Base de datos	Una BD compartida	BD propia por servicio
Tecnología	Stack único	Cada servicio elige
Fallo	Un bug puede bajar todo	Fallo aislado
Complejidad operativa	Baja	Alta (red, config, observ.)
Cuándo usarlo	Producto chico/inicial	Escala, equipos grandes

IMPORTANTE: los microservicios NO son siempre la solución correcta.

Agregan complejidad operativa real: red, configuración distribuida, observabilidad.
La elección depende del tamaño del equipo, la escala esperada y la madurez del dominio.
Para "Patitas Felices" lo hacemos para aprender el patrón — en producción real probablemente arrancaríamos con el monolito modulado.

3. Bounded Contexts del dominio veterinario

El primer paso para descomponer en microservicios es identificar los Bounded Contexts: fronteras del dominio donde los conceptos tienen significado propio y consistente.

Microservicio	Puerto	Entidad raíz	Responsabilidad
ms-dueno	8081	Dueno	CRUD de dueños + validaciones de negocio
ms-mascota	8082	Mascota	CRUD de mascotas + relación con dueño
ms-veterinario	8083	Veterinario	CRUD de veterinarios + especialidades
ms-turno	8084	Turno	Gestión de turnos + reglas de solapamiento

Regla de oro del Bounded Context:
Cada microservicio es dueño de sus datos y tiene su propia base de datos.
Nadie hace JOIN entre bases de datos de distintos microservicios.
La comunicación entre contextos se hace a través de APIs (HTTP o eventos).

4. Arquitectura Hexagonal (Ports & Adapters)

La arquitectura hexagonal organiza el código en tres zonas concéntricas:

ZONA INTERNA — DOMINIO

- Contiene las entidades y la lógica de negocio pura.
- No depende de Spring, JPA ni de ningún framework externo.
- Define los Ports: interfaces que describen QUÉ necesita el dominio (sin importar CÓMO).

ZONA MEDIA — APLICACIÓN

- Implementa los casos de uso.
- Orquesta el dominio sin contener lógica de negocio propia.
- Implementa los Ports de entrada (inbound ports).

ZONA EXTERNA — ADAPTADORES

- Adaptadores de entrada (inbound): Controllers REST, consumers de mensajería.
- Adaptadores de salida (outbound): implementaciones JPA, clientes HTTP, caches.
- Traducen entre el mundo externo y el lenguaje del dominio.

Capa	Paquete	Clases ejemplo

Domain	domain.model	Dueno.java (entidad JPA)
Domain	domain.port.in	DuenoService.java (interface)
Domain	domain.port.out	DuenoRepository.java (interface)
Application	application	DuenoServiceImpl.java
Adapters — in	adapter.in.web	DuenoController.java
Adapters — in	adapter.in.web.dto	DuenoRequest.java, DuenoResponse.java
Adapters — out	adapter.out.persistence	DuenoJpaRepository.java

La regla de dependencias (Dependency Rule):
Las dependencias siempre apuntan HACIA ADENTRO.
El dominio no conoce los adaptadores.
Los adaptadores conocen el dominio (a través de los ports).
Nunca al revés.

5. Estructura del proyecto Maven multi-módulo

En lugar de tener cuatro proyectos separados, usamos un proyecto Maven padre con módulos. Esto simplifica la gestión de dependencias y permite builds coordinados.

```
patitas-felices/              <- proyecto padre (pom.xml)
  ms-dueno/                   <- módulo 1
    src/main/java/com/patitas/dueno/
      domain/
        model/          Dueno.java
        port/
          in/           DuenoService.java (interface)
          out/          DuenoRepository.java (interface)
      application/      DuenoServiceImpl.java
      adapter/
        in/web/         DuenoController.java
          dto/          DuenoRequest.java, DuenoResponse.java
        out/persistence/  DuenoJpaRepository.java
    pom.xml
  ms-mascota/               <- módulo 2 (misma estructura)
  ms-veterinario/           <- módulo 3
  ms-turno/                 <- módulo 4
  pom.xml                   <- POM padre
```

6. Paso a paso — Implementación de ms-dueno

PASO 1 Crear el proyecto padre Maven

-> Crear la carpeta patitas-felices/ como raíz del proyecto.
-> Crear el pom.xml padre con packaging "pom" y los cuatro módulos declarados.
-> En <dependencyManagement> definir las dependencias comunes:
spring-boot-starter-web, spring-boot-starter-data-jpa, mysql-connector-j,
spring-boot-starter-validation, lombok, mapstruct.
-> Verificar con: mvn validate (desde la raíz).

PASO 2 Crear el módulo ms-dueno

-> Dentro de patitas-felices/ crear la carpeta ms-dueno/
-> Crear su pom.xml referenciando al padre con <parent>.
-> Crear la estructura de paquetes completa en src/main/java/com/patitas/dueno/
-> Crear src/main/resources/application.yml con la configuración de la BD.
IMPORTANTE: NO configurar spring.jpa.database-platform (Hibernate 7 lo detecta solo).
-> Puerto del servicio: server.port=8081
-> Nombre de la BD: dueno_db (crear en MySQL Workbench antes de levantar).

PASO 3 Implementar la capa Domain

-> Crear la entidad Dueno en domain/model/Dueno.java:
Campos: id (Long, @Id @GeneratedValue), nombre, apellido, email, telefono.
Anotar con @Entity, @Table(name="duenos"), @Data, @NoArgsConstructor,
@AllArgsConstructor.
-> Crear el port de entrada DuenoService en domain/port/in/DuenoService.java:
Interface con: findAll(), findById(Long), save(Dueno), update(Long, Dueno), deleteById(Long).
-> Crear el port de salida DuenoRepository en domain/port/out/DuenoRepository.java:
Interface con los mismos métodos.
-> Esta capa NO tiene anotaciones de Spring — es Java puro.

PASO 4 Implementar la capa Application

-> Crear DuenoServiceImpl en application/DuenoServiceImpl.java.
-> Debe implementar la interface DuenoService (port de entrada).
-> Debe recibir por constructor una instancia de DuenoRepository (port de salida).
-> Anotar con @Service.
-> Implementar cada método delegando al DuenoRepository.
-> En save(): validar que no exista un dueño con el mismo email.
Si existe, lanzar DuplicatedEmailException extends RuntimeException.
-> Verificar que DuenoServiceImpl NO importa nada de javax.persistence ni de Spring Web.

PASO 5 Implementar el adaptador de salida (Persistence)

-> Crear DuenoJpaRepository en adapter/out/persistence/DuenoJpaRepository.java.
-> Extender JpaRepository<Dueno, Long>.
-> Implementar la interface DuenoRepository (port de salida del dominio).
-> Anotar con @Repository.
-> Los métodos de JpaRepository satisfacen la mayoría de los ports.

-> Para la validación de email duplicado, agregar: boolean existsByEmail(String email).
Spring Data lo genera automáticamente.

PASO 6 Implementar el adaptador de entrada (Web)

-> Crear los DTOs en adapter/in/web/dto/:
DuenoRequest: campos con validaciones Bean Validation (@NotBlank, @Email, etc.).
DuenoResponse: campos expuestos al cliente (incluye id).
-> Crear DuenoController en adapter/in/web/DuenoController.java.
-> Anotar con @RestController, @RequestMapping("/api/duenos"), @RequiredArgsConstructor.
-> Inyectar DuenoService (la interface, NO la implementación).
-> Implementar: GET /, GET /{id}, POST /, PUT /{id}, DELETE /{id}.
-> Usar @Valid en los parámetros de request.
-> El controller NO conoce DuenoServiceImpl — solo la interface DuenoService.

PASO 7 Clase principal y primera ejecución

-> Crear MsDuenoApplication.java en com.patitas.dueno con @SpringBootApplication.
-> Verificar application.yml:
spring.application.name=ms-dueno
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/dueno_db
spring.jpa.hibernate.ddl-auto=update
(NO configurar spring.jpa.database-platform)
-> Levantar: mvn spring-boot:run -pl ms-dueno
-> Probar con Swagger: http://localhost:8081/swagger-ui/index.html
-> Verificar que todos los endpoints del Sprint 2 funcionan, ahora en puerto 8081.

PASO 8 Verificación de la arquitectura (Dependency Rule)

-> Revisar imports de DuenoServiceImpl — no debe importar nada de adapter.*
-> Revisar imports de DuenoController — no debe importar DuenoServiceImpl ni domain.model.*
(solo conoce los DTOs y la interface DuenoService).
-> Si sentís tentación de importar algo de una capa "más externa" desde una "más interna",
eso indica que algo está en el lugar incorrecto.
-> Documentar en el README la decisión: ¿por qué el controller no usa DuenoServiceImpl?

7. Criterios de aceptación del Sprint

El proyecto compila desde la raíz: mvn clean package -pl ms-dueno
ms-dueno levanta en puerto 8081 sin errores
Todos los endpoints CRUD de /api/duenos funcionan (verificar con Swagger)
La validación de email duplicado devuelve error descriptivo (no 500)
DuenoServiceImpl no importa nada del paquete adapter.*

DuenoController no importa DuenoServiceImpl (solo la interface)
El README tiene sección "Arquitectura" que explica la estructura de capas

8. Entregable del Sprint 9

- Repositorio GitHub con el proyecto patitas-felices/ (multi-módulo Maven).
- Módulo ms-dueno implementado con las tres capas hexagonales.
- Base de datos dueno_db en MySQL con tabla duenos creada por Hibernate.
- README con sección de arquitectura.
- Demostración funcional: POST y GET desde Swagger en localhost:8081.

9. Vista previa — Sprint 10

En el Sprint 10 vamos a:

- Replicar la estructura hexagonal en ms-mascota, ms-veterinario y ms-turno.
- Incorporar Eureka Server (Service Discovery).
- Incorporar Config Server para centralizar la configuración.
- Configurar Spring Boot Actuator para health checks.